

Password Attacks: Online & Offline

The attacker's reality

Most initial access does not need a 0-day – it needs a password that was re-used, default, or weak. Passwords are the #1 bridge between recon and foothold.

Online attacks (against a service)

Brute force: try everything (slow; rate-limits kill it)

Dictionary: try common words + mutations

Password spraying: ONE common password against MANY users (avoids lockout!)

Default creds: admin/admin inventory via the product's manual

```
hydra -L users.txt -p 'Welcome1' 10.0.0.30 smb      # spray one password
hydra -l admin -P rockyou.txt 10.0.0.20 ssh        # brute one user
hydra -L users.txt -P rockyou.txt 10.0.0.20 http-post-form "/login:user=^USER^&pass=^PASS^:F=incorrect"
```

Offline attacks (on the hash)

Dictionary: wordlist → hash word-by-word

Rule-based: mangling rules (leetspeak, suffixes, year append)

Mask: known structure (e.g., "Password2024!")

BM (brute): for very short/numeric

Rainbow tables: precomputed lookup (less relevant vs GPU crack)

GPU: hashcat on good hardware (millions-billions pass/sec)

```
hashcat -m 1000 -a 0 hashes.txt rockyou.txt        # NTLM
hashcat -m 1000 -a 0 hashes.txt rockyou.txt -r rules/best64.rule
hashcat -m 0 -a 3 hash.txt '?u?l?l?l?d?d'         # mask
```

Defense that actually works

MFA everywhere (defeats spraying instantly)

Long passphrases (≥ 12 -16 chars) over "complex" short ones

Lockout + alerting; but careful – spraying dodges lockout, so monitor

Password managers + unique per account

Hash-side: use salted slow hashes (bcrypt/argon2) so GPU cracking hits a wall

Password attacks win by statistics: the wordlist covers the 90% of users who choose "correct horse battery staple" poorly. Your job is to force the cost up and the pay-off down.

[← Module start](#)

[Credential Dumping & Pass-the-...](#)